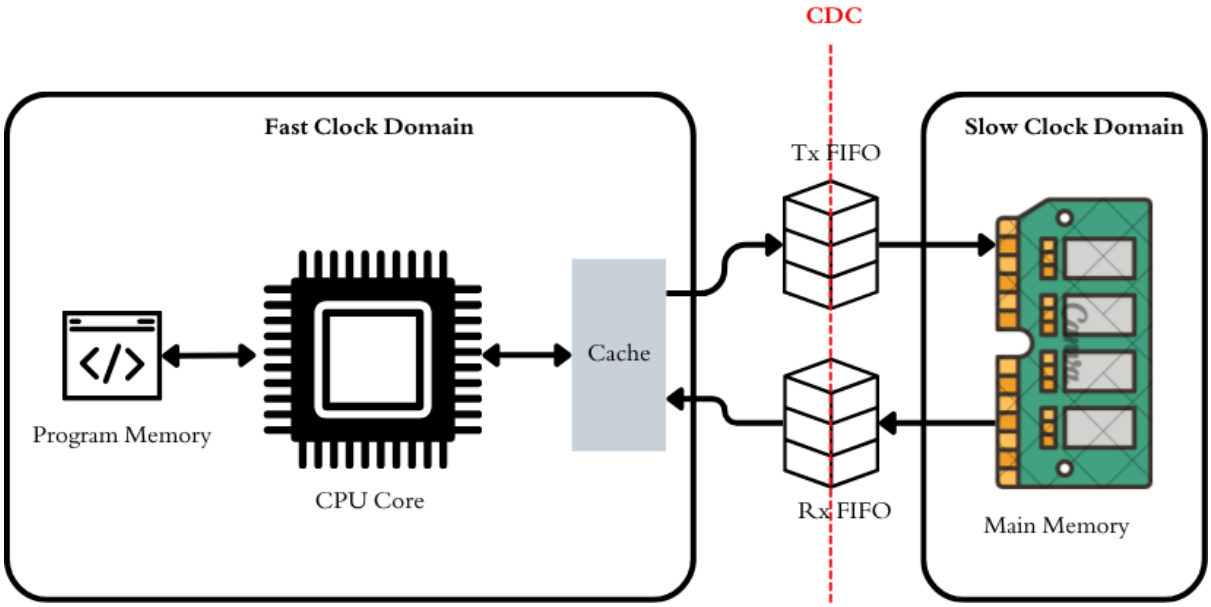
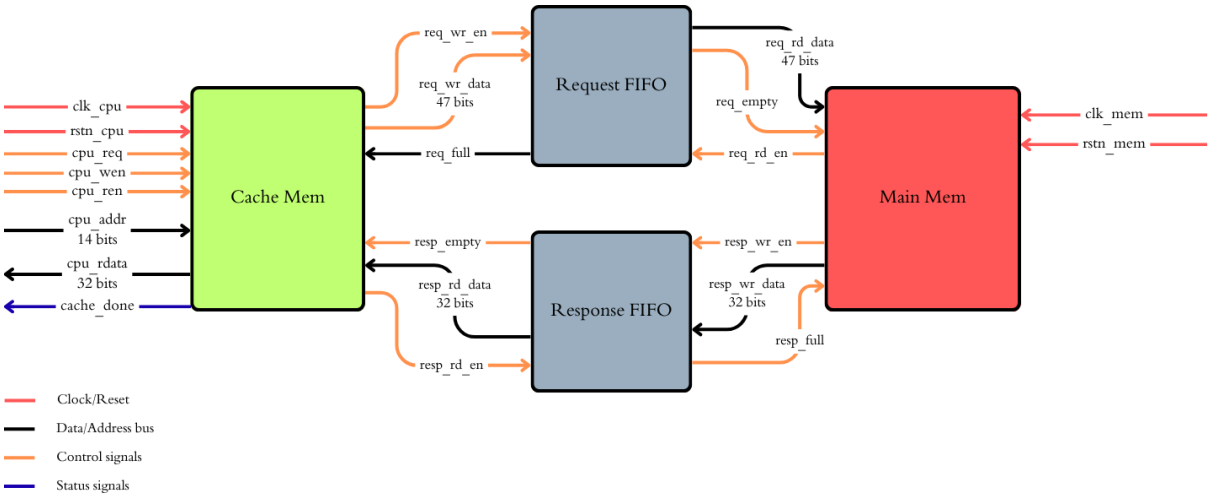
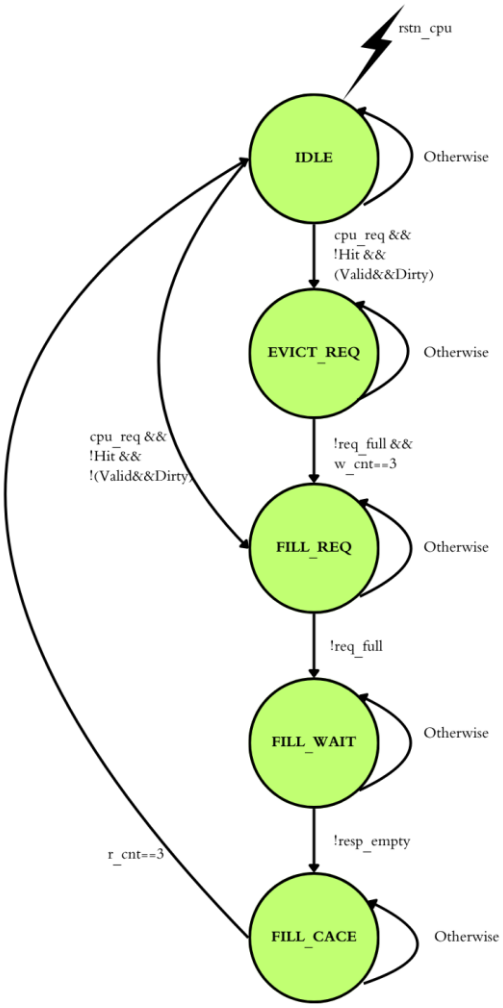
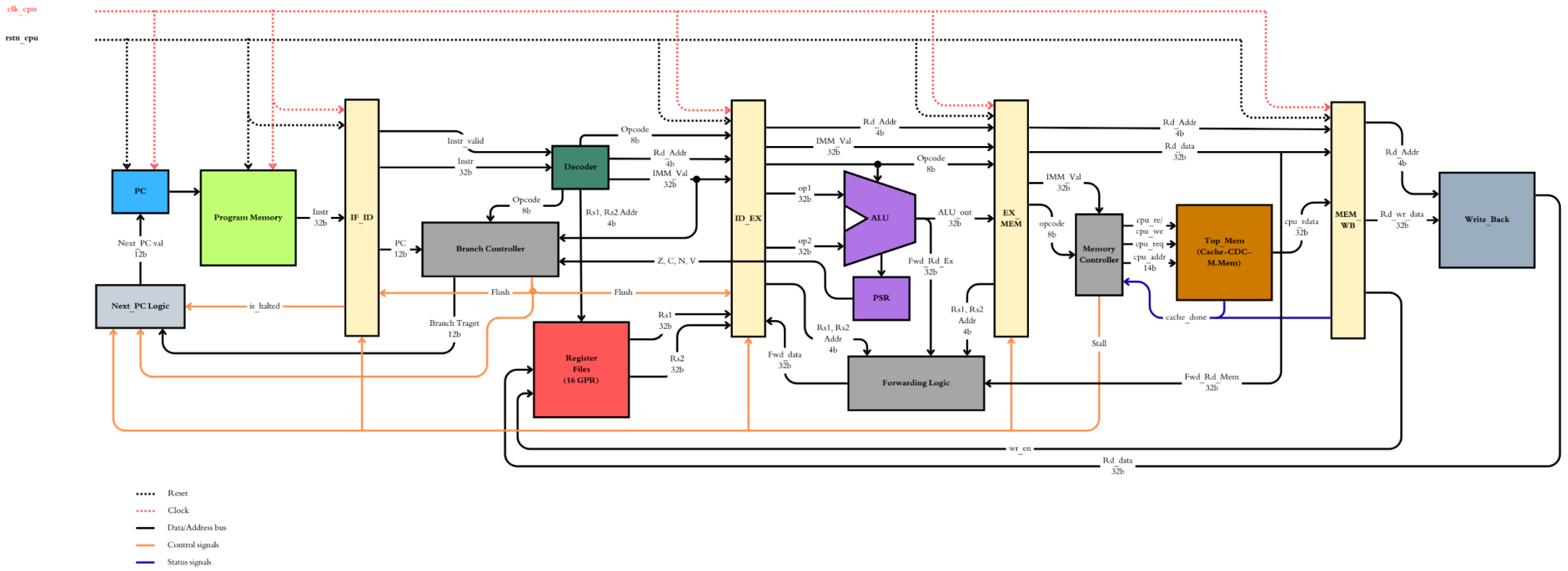


Top Memory Interface -- Micro Architecture:



Cache Memory FSM:




clk_cpu
rstn_cpu
PC
Program Memory
Instr 32b
IF_ID
Instr_valid
Instr 32b
Decoder
Opcode 8b
Rd_Addr 4b
IMM_Val 32b
Rs1, Rs2 Addr 4b
Next_PC val 12b
Next_PC Logic
is_halted
PC 12b
Branch Controller
Opcode 8b
Flush
Flush
Branch Traget 12b
Register Files (16 GPR)
Rs1 32b
Rs2 32b
ID_EX
Rd_Addr 4b
IMM_Val 32b
Opcode 8b
op1 32b
op2 32b
ALU
ALU_out 32b
PSR
Z, C, N, V
Fwd_Rd_Ex 32b
Rs1, Rs2 Addr 4b
Fwd_data 32b
Forwarding Logic
EX_MEM
Rd_Addr 4b
Rd_data 32b
IMM_Val 32b
opcode 8b
Memory Controller
Stall
cpu_re/ cpu_we
cpu_req
cpu_addr 14b
Top_Mem (Cache-CDC-M.Mem)
cache_done
cpu_rdata 32b
MEM_WB
Rd_Addr 4b
Rd_wr_data 32b
Write_Back
Fwd_Rd_Mem 32b
wr_en
Rd_data 32b
Reset
Clock
Data/Address bus
Control signals
Status signals

Instruction Set:

No Operation (Stall)

Mnemonic	Opcode	Fields used	Operation	Stages
NOP	00	—	No operation.	IF, ID

Load/Store Instructions

Mnemonic	Opcode	Fields used	Operation	Stages
LOAD	01	Rd, ADDR	Rd ← DATA[ADDR] (via cache)	All 5 Stages
LOAD_IND	02	Rd, Rs1	Rd ← DATA[Rs1] (via cache)	All 5 Stages
STORE	04	Rd(src), ADDR	DATA[ADDR] ← Rd (through cache)	IF,ID,EX,MEM
STORE_IND	05	Rd(src), Rs2	DATA[Rs2] ← Rd	IF,ID,EX,MEM

Register type Instructions

Mnemonic	Opcode	Fields used	Operation	Stages
ADD	06	Rd, Rs1, Rs2	Rd ← Rs1 + Rs2	IF,ID,EX,WB
SUB	07	Rd, Rs1, Rs2	Rd ← Rs1 − Rs2	IF,ID,EX,WB
MUL	08	Rd, Rs1, Rs2	Rd ← (Rs1 × Rs2)[31:0]	IF,ID,EX,WB
AND	09	Rd, Rs1, Rs2	Rd ← Rs1 & Rs2	IF,ID,EX,WB
OR	0A	Rd, Rs1, Rs2	Rd ← Rs1 | Rs2	IF,ID,EX,WB
NOT	0B	Rd, Rs1	Rd ← ~Rs1	IF,ID,EX,WB
XOR	10	Rd, Rs1, Rs2	Rd ← Rs1 ^ Rs2	IF,ID,EX,WB
CMP	0C	Rs1, Rs2	flags ← (Rs1 − Rs2); no write-back	IF,ID,EX,WB
EQ	0D	Rd, Rs1, Rs2	Rd ← (Rs1 == Rs2) ? 1 : 0	IF,ID,EX,WB
ADDI	16	Rd, Rs1, IMM	Rd ← Rs1 + sign_ext(IMM)	IF,ID,EX,WB
SUBI	17	Rd, Rs1, IMM	Rd ← Rs1 − sign_ext(IMM)	IF,ID,EX,WB
SHL	11	Rd, Rs1, Rs2	Rd ← Rs1 << Rs2[4:0] (logical)	IF,ID,EX,WB
SHR	12	Rd, Rs1, Rs2	Rd ← Rs1 >> Rs2[4:0] (logical)	IF,ID,EX,WB
SAR	13	Rd, Rs1, Rs2	Rd ← Rs1 >>> Rs2[4:0] (arithmetic)	IF,ID,EX,WB

ROL / ROR	14 / 15	Rd, Rs1, Rs2	Rotate left / right by Rs2[4:0]	IF,ID,EX,WB
ANDI / ORI / XORI	18 / 19 / 1A	Rd, Rs1, IMM	Logical op with zero_ext(IMM)	IF,ID,EX,WB
MOV	1B	Rd, Rs1	Rd ← Rs1	IF,ID,EX,WB
SLT / SLTU	1C / 1D	Rd, Rs1, Rs2	Rd ← (Rs1 < Rs2) ? 1 : 0 (signed / unsigned)	IF,ID,EX,WB
LUI	1E	Rd, IMM	Rd ← {IMM, 20'b0}	IF,ID,EX,WB
LOAD_IMM	03	Rd, IMM	Rd ← zero_ext(IMM)	IF,ID,EX,WB

Branch Instructions

Mnemonic	Opcode	Fields used	Operation	Stages
JMP	0E	ADDR	PC ← ADDR	IF, ID
JMP_IF	0F	Rs1, ADDR	if (Rs1 ≠ 0) PC ← ADDR	IF, ID
BEQ	20	ADDR	if Z: PC ← ADDR	IF, ID
BNE	21	ADDR	if ¬Z: PC ← ADDR	IF, ID
BLT	22	ADDR	if (N ⊕ V): PC ← ADDR (signed <)	IF, ID
BGE	23	ADDR	if ¬(N ⊕ V): PC ← ADDR (signed ≥)	IF, ID
BLTU / BGEU	24 / 25	ADDR	Unsigned < / ≥ branch (uses if C = 0, BLTU valid | if C = 1, BGEU valid)	IF, ID
JMP_REG	26	Rs1	PC ← Rs1[11:0] (computed jump / return)	IF, ID
CALL / RET	27 / 28	ADDR / —	Subroutine call / return (define a link register + stack)	IF, ID

Stop

Mnemonic	Opcode	Fields used	Operation	Stages
HALT	FF	—	Stop execution.	IF, ID

Instruction Bit Mapping:

Field	Bits	Width	Meaning
OPCODE	[31:24]	8	Operation code.
Rd / OUT	[23:20]	4	Destination register (also the data-source register for STORE).
Rs1 / OP1	[19:16]	4	First source register; base register for register-indirect addressing.
Rs2 / OP2	[15:12]	4	Second source register; shift amount uses Rs2[4:0].
IMM / ADDR	[11:0]	12	12-bit immediate or direct address.